

计算机操作系统课程设计详细设计文档

组长： 邵汉超 学号： 2023210932

组员及分工：

姓名	学号	分工
邵汉超	2023210932	消息总线与内核路由： 设计 MessageBus trait、LockedBus、Envelope、KernelMsg 枚举（7 变体）、Kernel 路由器（消息分发到 4 个服务 channel）
		进程管理服务： ProcessService 事件循环、PCB/TCB 进程控制块、SMP 调度器（5 种策略）、进程生命周期（fork/exec/exit/reap）、系统调用分发表（24 个 syscall → 17 个独立 handler）
		同步原语： 信号量 Semaphore（AtomicU32 CAS）、互斥锁 MutexLock（owner 跟踪）、TOCTOU 所有权转移、SyncManager
		内存管理与硬件模拟： MemoryService（分配/映射/换页）、FrameAllocator、SwapManager；VirtualCPU（20+ 指令、取指解码执行）、MMU（地址转换）、Timer（100Hz）
唐博艺	2023210942	文件系统与虚拟磁盘： FileService（消息驱动）、VFS 虚拟文件系统（inode 树 + JSON 持久化）、File 文件抽象（内存缓冲 + 磁盘映射）、FileDescriptorManager（每进程 fd 表）、VirtualDisk（512B 扇区）
		设备服务： DeviceService（直接订阅总线）、Device trait 与 DeviceRegistry、剪贴板设备（ClipboardGet/Set）、驱动管理 DriverManager
		Shell 与用户界面： 交互式命令行（20+ 命令）、pmon 进程监控器（TUI 实时显示）、minigdb 调试器（单步/寄存器/继续）
		用户态测试程序： syncdemo.asm、schedemo.asm、cmpdemo.asm 及 文件/设备操作.asm 程序

一、总体设计

1.1 概述

1.1.1 功能描述

genshin-OS (Chao-OS) 是一个基于 Rust 语言开发的微内核操作系统模拟器。系统严格遵循四层架构（用户交互层、交换层、服务层、硬件层），所有跨模块通信均通过中央异步消息总线完成。项目实现了完整的进程管理（Fork/Exec/Exit/Wait 模型）、SMP 双核调度、信号量与互斥锁同步、文件系统 CRUD、虚拟磁盘持久化、设备管理、以及一个交互式命令行 Shell。

1.1.2 运行环境

- 操作系统：Linux（Arch Linux / Ubuntu 22.04+）
- 运行时：Rust 编译的二进制文件，无外部运行时依赖
- 终端：支持 ANSI 转义序列的终端模拟器

1.1.3 开发环境

- 编程语言：Rust（edition 2021）
- 编译器：rustc 1.70+
- 构建工具：Cargo
- 版本控制：Git
- 编辑器：VS Code / Neovim

1.2 设计思想

1.2.1 软件设计构思

本系统采用微内核架构，核心思想是最小化内核态功能，最大化用户态服务。内核（Kernel）仅负责消息路由，不执行任何业务逻辑。所有系统功能（进程管理、内存管理、文件系统、设备管理）均作为独立的用户态服务运行，彼此之间通过消息总线（MessageBus）进行异步通信。

系统模拟了真实的硬件层：VirtualCPU 执行自定义 Mock ISA 指令，MMU 负责虚拟地址到物理地址的转换，Timer 以 100Hz 频率发出时钟中断驱动进程调度，VirtualDisk 提供基于文件的块设备存储。

进程管理严格遵循 Unix 的 Fork-Exec-Exit-Wait 模型，不使用“fire and forget”的 Spawn 模式。所有程序（包括 ls、mkdir 等内置命令）均通过 Fork + Exec + Wait 流程运行在独立的虚拟 CPU 上。

1.2.2 关键技术与算法

- (1) **SMP 多核调度**: 双 CPU 独立调度, RR 时间片轮转算法 (quantum=3 ticks), dequeue_next 跳过已在其他 CPU 运行的 PID, 每 CPU 维护 last_running 状态
- (2) **TOCTOU 所有权转移**: 信号量释放时不 count++, 而是扫描进程表找到 Blocked 等待者, 直接转移所有权。避免多核环境下调用者立即抢回信号量的竞态
- (3) **CAS 无锁同步**: 信号量和互斥锁的 wait/signal 操作使用 AtomicU32/U64 的 compare_exchange_weak 实现, 无需互斥锁保护
- (4) **VFS inode 树**: 基于 HashMap 的 inode 表 + 父目录 children HashMap 实现路径解析和文件查找, 支持 JSON 持久化
- (5) **系统调用直接路径**: INT 0x80 触发的中断不经过消息总线, 直接由 ProcessService 的 handle_file_syscall 在定时器中断步进循环中处理, 避免 SyscallTrap 消息泛滥

1.2.3 基本数据结构

全局核心数据结构包括:

- **KernelMsg**: 所有消息的统一枚举, 包含 Syscall、Interrupt、Process、Memory、File、Device 七个变体, 是整个系统的通信语言
- **PCB(Process Control Block)**: 进程控制块, 记录进程状态(Creating→Ready→Running→Blocked)、页表、CPU 状态、父进程、打开文件等
- **VFSNode**: 虚拟文件系统节点, 包含 inode、类型、权限、父子关系、数据块列表
- **Semaphore / MutexLock**: 基于原子变量的同步原语, Semaphore 用 AtomicU32 计数, MutexLock 用 AtomicU64 跟踪持有者
- **Envelope**: 消息信封, 封装 KernelMsg + 请求 ID + 响应通道, 支持 fire-and-forget 和 request-response 两种通信模式

1.3 基本处理流程

系统的核心处理流程是时钟中断驱动的调度循环:

- (1) Timer 以 100Hz 频率向消息总线发送 Interrupt::Timer

- (2) Kernel 路由器将中断转发到 ProcessService 的 intr_rx 通道
- (3) ProcessService 的主循环收到中断后调 handle_timer_interrupt()
- (4) 遍历两个虚拟 CPU：每个 CPU 先调 schedule(cpu_id) 从就绪队列选进程，再执行最多 3 条指令
- (5) 指令执行过程中若遇到 INT 0x80，直接调用 handle_file_syscall 分发系统调用
- (6) 检查僵尸进程 → 回收 → 循环

文件系统操作流程：用户进程执行 INT 0x80 (R0=18 listdir) → ProcessService 转发 FileRequest::ListDir 到消息总线 → Kernel 路由到 FileService → FileService 调 VFS.lookup_path 解析路径 → 收集子节点名称 → 通过响应通道返回 → ProcessService 将结果写入进程虚拟内存 0x200 → 进程通过 INT 0x80 (R0=2) 打印结果。

1.4 软件的体系结构设计

1.4.1 软件体系结构框图



1.4.2 软件主要模块及其依赖关系说明

- **Kernel**：唯一订阅 MessageBus 的组件。根据 KernelMsg 类型将消息路由到 ProcessService、MemoryService、FileService 的专用 channel。DeviceService 直接订阅总线，不经 Kernel 路由。

- **ProcessService**: 最复杂的服务, 依赖 SyncManager (同步原语)、IPCManager (消息队列/共享内存)、Scheduler (就绪队列管理)、MMU (地址转换)。接收 ProcessRequest 和 Syscall 两类消息。
- **FileService**: 依赖 VFS (目录树)、FileDescriptorManager (fd 管理)、VirtualDisk (块存储)。通过 VFS 的 JSON 序列化实现持久化。
- **MemoryService**: 管理 PhysicalMemory, 依赖 MMU 完成页表操作和地址映射。
- **DeviceService**: 独立订阅总线, 管理剪贴板等设备, 通过 DeviceRegistry 注册设备驱动。
- **Shell**: 运行在主线程, 通过 UIContext.send_and_wait 与消息总线交互, 不依赖任何服务。

1.5 软件数据结构设计

1.5.1 全局数据结构说明

系统中最关键的全局数据结构是 KernelMsg 枚举(定义在 src/messaging/msg.rs:27), 它是所有模块间通信的唯一载体。定义如下:

```
1 pub enum KernelMsg {
2     Syscall(Syscall),           // 用户系统调用 (CreateProcess,
    ExitProcess 等)
3     Interrupt(Interrupt),       // 硬件中断 (Timer, PageFault,
    SyscallTrap 等)
4     Process(ProcessRequest),    // 进程服务请求 (调度/IPC/同步/生命
    周期)
5     Memory(MemoryRequest),      // 内存服务请求 (分配/映射/换页)
6     File(FileRequest),         // 文件服务请求 (CRUD/目录操作)
7     Device(DeviceRequest),      // 设备服务请求 (ClipboardGet/Set)
8 }
```

ProcessRequest 枚举包含 30+ 变体, 覆盖进程调度 (Schedule/Block/Unblock)、IPC (SendMessage/ReceiveMessage/CreateSharedMemory)、同步 (CreateSemaphore/WaitSemaphore)、生命周期 (ForkProcess/ExecProcess/WaitChild/Kill) 等所有进程相关操作。

1.5.2 数据结构与系统单元的关系

	ProcessService	FileService	MemoryService	Shell
PCB（进程控制块）	✓			
VFSNode（文件节点）		✓		
Semaphore/MutexLock	✓			
PageTable（页表）	✓		✓	
FileDescriptor		✓		
KernelMsg（消息）	✓	✓	✓	✓
VirtualDisk（磁盘）		✓	✓	

1.6 软件接口设计

(1) 操作界面（命令接口）

用户通过交互式 Shell 输入命令。Shell 解析命令后通过消息总线发送 Process-Request 或 FileRequest，等待服务响应（3 秒超时）。支持的命令包括：ls、mkdir、touch、cat、write、rm、tree、stat（文件操作）、ps/pstree（进程管理）、kill（信号）、dual（多进程演示）、pmon（监控）、minigdb（调试）、cd/pwd（导航）、verbose（调试开关）、uptime（时钟计数）、help。

(2) 系统功能服务界面（程序接口）

用户进程通过 INT 0x80 软中断请求系统服务。R0 寄存器存放系统调用号，R1/R2 存放参数。ProcessService 的 handle_file_syscall 函数根据 R0 值分派到 24 个独立 handler。文件类系统调用（R0=10-18）将结果写入进程虚拟内存 0x200，进程通过 R0=2（print_str）打印结果。

1.6.1 外部接口

输入：键盘输入命令，通过 stdin 读取；程序文件从 programs/ 目录加载.asm 源文件，由汇编器（src/services/process/assembler.rs）转换为字节码。

输出：屏幕显示通过 stdout 输出 Shell 提示符、命令执行结果、进程输出(PRINT)、调试信息。文件数据持久化到.genshin-vfs.json（VFS 结构）和.genshin-disk.img（虚拟磁盘内容）。

1.6.2 内部接口

消息总线:所有服务间通信的唯一通道。发送方调用 `bus.send(msg)` 或 `bus.send_request(msg)`, 消息通过 Kernel 路由到目标服务的接收 channel。请求-响应模式下, Envelope 携带 `response_channel`, 服务通过 `envelope.respond_success()` 回复。

进程虚拟内存约定: 执行 `exec` 时, 程序路径写入虚拟地址 `0x100`, 参数写入 `0x200`。文件系统调用的返回数据写入 `0x200`, 字节数写入 CPU 的 R2 寄存器。

二、用户界面设计

2.1 界面的关系图

系统提供两种用户界面：

- (1) **交互式 Shell**：启动后进入命令行界面，显示欢迎信息和提示符 `genshin-os:/>`。用户输入命令后，Shell 解析并通过消息总线发送请求，同步等待响应后打印结果。
- (2) **pmon 进程监控器**：输入 `pmon` 命令进入 TUI 界面，实时显示进程状态表、内存使用、磁盘信息。按 `q` 退出。

2.2 界面说明

2.2.1 Shell 命令行界面

启动系统后，终端显示：

```
1 Initializing Genshin-OS microkernel simulation...
2   Hardware + Message bus
3   Kernel
4   Timer (hardware, 100 Hz)
5   Process service
6   Memory service
7   File service
8   Device service
9   Shell
10
11
12           Welcome to Genshin-OS Microkernel Shell
13           Version 0.2.0
14
15   Type 'help' for available commands or 'exit' to quit.
16
17
18 genshin-os:/>
```

常用命令示例：

- `ls` — 列出当前目录
- `mkdir testdir` — 创建目录
- `touch testdir/a.txt` — 创建文件

- `write testdir/a.txt hello` —写入文件
- `cat testdir/a.txt` —读取文件
- `ps` —查看进程树（状态、PID、名称）
- `dual syncdemo` —启动两个互斥演示进程
- `verbose on` —开启详细调试输出
- `pmon` —打开进程监控器
- `minigdb ls` —单步调试 `ls` 程序

三、相关处理流程

3.1 程序单元 1：消息总线与内核路由

3.1.1 程序单元说明

消息总线是整个系统的通信中枢。本单元实现 MessageBus trait 及其 LockedBus 实现、Envelope 信封机制、KernelMsg 统一消息枚举、以及 Kernel 路由器。所有跨模块通信均通过消息总线，无直接函数调用。

3.1.2 数据结构说明

- **MessageBus trait**(src/messaging/bus.rs:96): 定义 send(发后即忘)、send_request(请求-响应)、subscribe(订阅) 三个核心方法
- **LockedBus**(src/messaging/bus.rs:159): 基于 Arc<Mutex<Vec<Sender<Envelope>>> 的实现，send 遍历订阅者广播，subscribe 创建 unbounded channel
- **Envelope**(src/messaging/bus.rs:31): 封装 message + request_id + response_channel, 支持 fire_and_forget 和 with_response 构造
- **KernelMsg** (src/messaging/msg.rs:27): 7 变体统一消息枚举
- **Kernel** (src/services/kernel.rs): 持有 4 个 Sender (process/intr/memory/file), 按消息类型路由

3.1.3 算法及流程

消息发送流程（以 Shell 执行 ls 命令为例）：

- (1) Shell 调用 context.send_request(KernelMsg::Process(ForkProcess{parent_pid:1}))
- (2) LockedBus::send_request 创建 Envelope (含 response_channel)，遍历 subscribers 用 try_send 广播
- (3) Kernel 的 receiver.recv() 收到消息，route() 判断消息类型为 Process → process_tx.send(envelope)
- (4) ProcessService 的 receiver.try_recv() 收到消息，handle_envelope 处理 ForkProcess
- (5) ProcessService 通过 envelope.respond_success(Pid(child_pid)) 发送响应
- (6) Shell 的 rx.recv_timeout(3s) 收到响应 → 继续发送 ExecProcess

3.1.4 数据存储说明

无需数据存储。消息在内存中的 unbounded channel 传输，不持久化。

3.1.5 源程序文件说明

- `src/messaging/bus.rs` —MessageBus trait、LockedBus、DirectBus、Envelope
- `src/messaging/msg.rs` —KernelMsg、ProcessRequest、FileRequest、IPCMessage 等枚举
- `src/messaging/response.rs` —Response、ResponseData、ServiceError
- `src/messaging/mod.rs` —模块导出
- `src/services/kernel.rs` —Kernel 路由器

3.1.6 函数说明

- `LockedBus::send(msg)` —遍历所有订阅者, `try_send` 广播消息, fire-and-forget
- `LockedBus::send_request(msg)` —创建 Envelope + response_channel, 广播, 返回 `Receiver<Response>`
- `LockedBus::subscribe()` —创建 unbounded channel, 将 Sender 加入 subscribers, 返回 `Receiver`
- `Envelope::fire_and_forget(msg)` —创建无响应通道的信封
- `Envelope::with_response(msg)` —创建含响应通道的信封
- `Kernel::route(envelope)` —根据 KernelMsg 变体将消息发送到对应服务的 Sender

3.2 程序单元 2: 进程管理服务与调度器

3.2.1 程序单元说明

ProcessService 是系统中最核心的服务, 负责进程的完整生命周期管理 (创建、加载、运行、阻塞、退出、回收)、SMP 双核调度、IPC 通信、同步原语操作、以及系统调用分发。

3.2.2 数据结构说明

- **PCB** (src/services/process/pcb.rs:261): 进程控制块, 包含 pid、name、state (状态机)、parent_pid、priority、threads
- **TCB** (src/services/process/pcb.rs:108): 线程控制块, 包含 tid、entry_point、stack_pointer、state
- **Scheduler**(src/services/process/scheduler.rs:45): 就绪队列 VecDeque<ReadyQueueEntry>, per-CPU cpu_current 和 cpu_ticks, 支持 RR/FIFO/Priority/SJF/MLFQ
- **IPCManager**(src/services/process/ipc.rs:240): 消息队列 HashMap<Pid, MessageQueue>, 共享内存 HashMap<shmid, SharedMemoryRegion>
- **SyncManager** (src/services/process/sync.rs:389): 信号量和互斥锁的集中管理器, 预创建全局 semaphore 0 和 mutex 0

3.2.3 算法及流程

SMP 调度算法 (handle_timer_interrupt, service.rs:571-740):

- (1) 每 tick 遍历两个 CPU (cpu_id = 0, 1)
- (2) schedule(cpu_id): 检查当前进程量子是否用完 (3 ticks), 用完则重新入队; 从就绪队列取下一个, 跳过已在其他 CPU 运行的 PID
- (3) 状态机: 将上次运行的进程从 Running 切换为 Ready, 将新进程标记为 Running
- (4) CPU 步进循环: 最多执行 3 条指令, 遇 INT 0x80 直接调用 handle_file_syscall
- (5) Zombie 检查: 若 CPU 停机的进程并非 Blocked, 则标记为 Zombie, 释放信号量
- (6) Reaper: 每 tick 扫描一个 Zombie 进程并回收 (释放内存、移除 PCB)

进程创建流程 (fork+exec, 非 Spawn):

- (1) Shell 发送 ForkProcess{parent_pid: 1} → fork_impl 克隆父进程页表 → 创建子进程 PCB/CPU → **不立即调度** (等待 exec 加载代码)
- (2) Shell 发送 ExecProcess{pid, executable} → exec_impl 加载.asm 程序 → 写虚拟内存 → 重置 CPU PC=0 → 加入就绪队列
- (3) 这种方式避免了子进程在代码加载前被调度执行导致页错误/僵尸的问题

3.2.4 数据存储说明

进程控制块和调度器状态均为内存数据结构，不持久化。进程退出时页帧被回收。

3.2.5 源程序文件说明

- `src/services/process/service.rs` —ProcessService 主逻辑（事件循环、系统调用分发表、定时器中断处理）
- `src/services/process/pcb.rs` —PCB/TCB 进程控制块
- `src/services/process/scheduler.rs` —SMP 调度器
- `src/services/process/ipc.rs` —IPCManager、MessageQueue、SharedMemoryRegion
- `src/services/process/sync.rs` —Semaphore、MutexLock、SyncManager
- `src/services/process/assembler.rs` —.asm 汇编器

3.2.6 函数说明

- `handle_timer_interrupt()` —定时器中断处理，SMP 调度 + CPU 步进 + Zombie 回收（`service.rs:571-740`）
- `fork_impl(parent_pid)` —克隆父进程的内存和 CPU 状态（`service.rs:973-1050`）
- `exec_impl(pid, executable, args)` —加载程序、替换页表、重置 CPU（`service.rs:1100-1151`）
- `handle_file_syscall(cpu, r0, r1, r2)` —系统调用分发表，17 行 match 分发到 24 个独立 handler（`service.rs:1450-1486`）
- `Scheduler::schedule(cpu_id)` —SMP 调度决策（`scheduler.rs:127-142`）
- `Semaphore::wait()` —CAS 循环实现 P 操作（`sync.rs:82-107`）
- `MutexLock::try_acquire(pid)` —CAS 实现获取锁（`sync.rs:276-306`）

3.3 程序单元 3：同步原语

3.3.1 程序单元说明

本单元实现了两个核心同步原语：信号量（Semaphore）和互斥锁（MutexLock）。两者均基于 Rust 的 AtomicU32/AtomicU64 实现无锁并发操作。关键的 TOCTOU 所有权转移机制解决了多核环境下信号量释放后立即被调用者抢回的竞态问题。

3.3.2 数据结构说明

- **Semaphore** (sync.rs:25): value (AtomicU32 计数器)、max_value、wait_count、owner_pid、valid 标志
- **MutexLock** (sync.rs:218): owner (AtomicU64, u64::MAX= 未锁定)、count (递归计数)、recursive 标志、wait_count
- **SyncManager** (sync.rs:389): HashMap<id, Semaphore> + HashMap<id, MutexLock>, 集中管理所有同步原语

3.3.3 算法及流程

Semaphore::wait() P 操作 (sync.rs:82-107):

```

1 pub fn wait(&self) -> SemaphoreResult {
2     let mut current = self.value.load(Ordering::Acquire);
3     loop {
4         if current == 0 {
5             self.wait_count.fetch_add(1, Ordering::AcqRel);
6             return SemaphoreResult::WouldBlock;
7         }
8         match self.value.compare_exchange_weak(
9             current, current - 1, Ordering::AcqRel, Ordering::
10                Acquire
11            ) {
12             Ok(_) => return SemaphoreResult::Acquired,
13             Err(new) => current = new,
14         }
15     }
16 }
```

TOCTOU 所有权转移 (service.rs:1668-1693): 当进程调用 sem_signal 时, 不直接 count++, 而是扫描进程表查找 Blocked(WaitingForLock{lock_addr == sem_id}) 的等待者。找到等待者时:

- (1) PCB 状态设为 Ready
- (2) scheduler.ready(wpid) 加入就绪队列
- (3) cpu.halted = false 唤醒 CPU
- (4) 信号量 count 保持 0 (不增加) ——所有权直接转移给等待者

如果未找到等待者, 则 sem.signal() → count++。

3.3.4 源程序文件说明

- `src/services/process/sync.rs` —全文 (Semaphore、MutexLock、SyncManager)
- `src/services/process/service.rs:1642-1752` —同步系统调用 handler

3.4 程序单元 4: 文件系统与虚拟磁盘

3.4.1 程序单元说明

FileService 是文件系统的顶层服务,整合了 VFS 目录树、File 文件抽象、FileDescriptorManager 文件描述符管理、VirtualDisk 虚拟磁盘四个层次。所有文件操作通过消息总线接收 FileRequest, 由 FileService 协调下层组件完成。

3.4.2 数据结构说明

- **VFSNode** (vfs.rs:24): inode、node_type (File/Directory/SymLink)、name、parent、permissions、size、blocks (磁盘块号列表)、children (子节点 HashMap)
- **VirtualFileSystem**(vfs.rs:173): nodes: HashMap<inode, VFSNode>, next_inode: 自增分配器, mounts: 挂载点表
- **File** (file.rs:100): data: Vec<u8> 内存缓冲、position: 读写光标、start_sector: 磁盘起始位置、dirty: 脏标记
- **OpenFile** (descriptor.rs:18): fd + owner + file 引用 + flags
- **FileDescriptorManager** (descriptor.rs:150+): 每进程 fd 表, alloc/dealloc 管理
- **VirtualDisk**(disk.rs:30+): 512B 扇区, read_sector/write_sector, 基于.genshin-disk.img 文件

3.4.3 算法及流程

文件读取完整流程 (cat /foo/bar.txt):

- (1) cat.asm: MOV R0,#10 → INT 0x80 (open, 路径从 0x100 读取)
- (2) ProcessService → bus.send(FileRequest::Open{path, flags}) → FileService
- (3) FileService: vfs.lookup_path(path) 递归解析路径 → 获取 inode → 查找或创建 File → fd_manager.alloc(pid, file) → 返回 fd
- (4) cat.asm: MOV R0,#12 → INT 0x80 (read)

(5) FileService: `fd_manager.get(fd) → OpenFile.read(count) → File.read(count) →`
从 `data[position..]` 切片返回数据

(6) ProcessService: 将返回数据写入进程虚拟内存 0x200, 设 R2= 字节数

(7) cat.asm: `MOV R0,#2 → INT 0x80` (`print_str`, 从 0x200 打印 R2 字节)

VFS 持久化:每次文件操作后,FileService 调用 `vfs.save_to_file(".genshin-vfs.json")` 将整个 VFS 树序列化为 JSON 写入磁盘。启动时调用 `VirtualFileSystem::load_from_file` 恢复。

3.4.4 数据存储说明

- `.genshin-vfs.json` —VFS 树结构的 JSON 持久化文件, 包含所有节点信息 (inode、类型、父子关系、权限、磁盘块号)
- `.genshin-disk.img` —虚拟磁盘镜像文件, 512B 扇区, 存储文件实际数据

3.4.5 源程序文件说明

- `src/services/file/service.rs` —FileService 主逻辑
- `src/services/file/vfs.rs` —VFS 虚拟文件系统
- `src/services/file/file.rs` —File 文件抽象
- `src/services/file/descriptor.rs` —文件描述符管理
- `src/hardware/disk.rs` —VirtualDisk 虚拟磁盘
- `src/hardware/block.rs` —块设备层

3.5 程序单元 5: Shell 与用户界面

3.5.1 程序单元说明

Shell 运行在主线程, 提供交互式命令行界面。解析用户输入后通过消息总线发送请求, 同步等待服务响应并打印结果。包含 `pmon` 进程监控器和 `minigdb` 调试器两个辅助工具。

3.5.2 数据结构说明

- **Shell** (`shell/mod.rs:38`): 持有 `UIContext` (封装 `MessageBus`)、`Timer` 引用、当前工作目录 `cwd`
- **UIContext** (`ui/mod.rs:18`): 封装 `MessageBus`, 提供 `send(msg)` 和 `send_request(msg) → Receiver<Response>`

- **Command** (shell/parser.rs): 解析后的命令结构, 包含 name 和 args

3.5.3 算法及流程

Shell 命令执行流程 (以 ls 为例):

- (1) 用户输入 ls → Parser 解析为 Command{name: "ls", args: []}
- (2) execute_command() → 匹配"ls" → fork_exec_wait("ls", &["/"])
- (3) fork_exec_wait 内部: send_and_wait(ForkProcess) → 获取子进程 PID → send_and_wait(ExecProcess{pid, "ls"}) → send_and_wait(WaitChild{pid, child_pid})
- (4) ProcessService 处理 Fork → Exec (加载 ls.asm) → 子进程运行 → MOV R0,#18 → INT 0x80 → 结果写入 0x200 → MOV R0,#2 → 打印结果 → HALT
- (5) ProcessService 收到 WaitChild, 等待子进程退出, 回收资源
- (6) Shell 收到响应, 打印结果

3.5.4 源程序文件说明

- src/ui/shell/mod.rs —Shell 主逻辑(命令解析、执行、fork_exec_detach/wait、minigdb)
- src/ui/shell/parser.rs —命令解析器
- src/ui/shell/builtins.rs —内置命令
- src/ui/shell/pmon.rs —进程监控器 TUI
- src/ui/mod.rs —UIContext

3.6 程序单元 6: 硬件模拟层

3.6.1 程序单元说明

硬件模拟层提供了操作系统运行所需的虚拟硬件: VirtualCPU 执行自定义 Mock ISA 指令、MMU 管理虚拟地址到物理地址的映射、Timer 以 100Hz 频率驱动进程调度、PhysicalMemory 和 VirtualDisk 提供存储。

3.6.2 数据结构说明

- **VirtualCPU**(cpu.rs:260+): registers[4], pc, sp, flags(Zero/Sign/Carry/Overflow), halted, syscall_pending, syscall_regs
- **MMU**(mmu.rs): 管理 per-PID 的页表, map_page/unmap_page/translate/get_page_entries
- **Timer**(timer.rs:51): tick_interval(10ms), tick_count, state(Stopped/Running/Paused), 通过独立线程发送 Interrupt::Timer
- **PhysicalMemory** (memory.rs): 4MB 大数组模拟物理内存

3.6.3 算法及流程

CPU 指令执行流程 (cpu.rs:371-404):

- (1) fetch_instruction(): 从 MMU 读取 8 字节 (opcode + dst + src_type + pad + value)
- (2) 解码 opcode(0x01=MOV, 0x03=SUB, 0x10=JMP, 0x13=JNZ, 0x80=INT, 0xFF=HALT 等)
- (3) execute_instruction(): 执行指令, 更新寄存器和标志位
- (4) INT 0x80 特殊处理: 保存寄存器到 syscall_regs, 设 syscall_pending=true, 不抛异常

定时器驱动调度(timer.rs:107-142): 独立线程每 10ms 循环 → thread::sleep(10ms) → bus.send(Interrupt::Timer) → Kernel 路由到 ProcessService 的 intr_rx → handle_timer_interrupt() 执行一轮调度。

3.6.4 源程序文件说明

- src/hardware/cpu.rs —VirtualCPU (取指、解码、执行、中断处理)
- src/hardware/mmu.rs —MMU 地址转换
- src/hardware/timer.rs —Timer 定时器
- src/hardware/memory.rs —PhysicalMemory 物理内存
- src/hardware/disk.rs —VirtualDisk 虚拟磁盘

四、总结

genshin-OS 从零实现了一个微内核操作系统的核心组件。通过本项目，我们深入理解了以下操作系统核心概念：

- (1) **微内核架构**：内核只做消息路由，所有服务在用户态运行，通过消息总线通信。这种设计带来了良好的模块化和可扩展性，但也引入了消息传递的开销。
- (2) **SMP 多核调度**：实现双 CPU 独立调度的过程中，遇到了 PID 重复分配、就绪队列竞态、量子过期处理等多个实际问题，加深了对调度器设计的理解。
- (3) **同步原语与竞态**：CAS 无锁实现的信号量和互斥锁在高并发场景下表现出正确性。TOCTOU 所有权转移机制是解决多核环境下锁释放竞态的关键设计。
- (4) **虚拟文件系统**：VFS 的 inode 树 + JSON 持久化方案在简单性和可用性之间取得了良好平衡。路径解析、文件描述符管理、磁盘块映射构成了完整的文件系统栈。
- (5) **系统调用机制**：INT 0x80 → syscall_pending → handle_file_syscall 分发的两层译码模型，在保持硬件层纯粹性的同时实现了高效的系统调用处理。

项目开发过程中遇到的主要挑战包括：多核调度中进程被误判为 Zombie 导致提前回收（通过 fork 后不立即调度的策略修复）、信号量 count 在进程退出时被无条件 inflate 导致互斥失效（通过 count==0 前置检查和 holder 跟踪设计逐步改进）、消息总线的 SyscallTrap 泛滥导致服务无响应（通过外循环 Timer-only 过滤和内循环移除修复）等。这些问题的调试过程加深了我们对并发编程和操作系统内核设计复杂性的认识。

五、附录

5.1 源码文件清单

目录/文件	说明
src/main.rs	主入口，启动所有线程
src/messaging/	消息总线 (bus.rs、msg.rs、response.rs)
src/services/kernel.rs	Kernel 路由器
src/services/process/	进程管理 (service.rs、pcb.rs、scheduler.rs、ipc.rs、sync.rs、assembler.rs)
src/services/memory/	内存管理 (service.rs、alloc.rs、paging.rs、swap.rs)
src/services/file/	文件系统 (service.rs、vfs.rs、file.rs、descriptor.rs)
src/services/device/	设备管理 (service.rs、device.rs、driver.rs、manager.rs)
src/hardware/	硬件模拟 (cpu.rs、mmu.rs、timer.rs、memory.rs、disk.rs、block.rs、device.rs)
src/ui/shell/	Shell (mod.rs、parser.rs、builtins.rs、pmon.rs)
src/ui/mod.rs	UIContext
programs/	用户态.asm 测试程序
docs/	设计文档 (ipc.md、syscalls.md、syncdemo.md 等)

5.2 参考文献

- [1] Rust 编程语言官方文档: <https://doc.rust-lang.org/book/>
- [2] xv6 教学操作系统: <https://pdos.csail.mit.edu/6.828/2023/xv6.html>
- [3] seL4 微内核: <https://sel4.systems/>
- [4] 操作系统概念 (第 10 版), Abraham Silberschatz 等
- [5] The Design of the UNIX Operating System, Maurice J. Bach